Claude Community Berlin · The Delta Campus · 30 October

Ship a live website in one evening.

Hooman Abbasi

Founder and Design Director, Raise Studio

Hi, I'm Hooman.

First

Software engineer

I learned how to build.

Then

Designer

I learned how to design.

Now

Founder

I learned everything else: marketing, sales, all of it.

Your website does a lot of your selling.

When people land on it, they need to understand what you do in seconds. And it has to look like you take your work seriously.

Every way I've built a website.

01

Hand-coded

Built from scratch, self-hosted.

02

WordPress

Easier to manage the content.

03

Webflow and Framer

What we still use for clients every week.

04

With Claude

Tonight.

"With AI" doesn't mean the site runs on AI. It means AI helps us build it.

Part 1 · Context

So why Claude, if we already use Webflow and Framer?

Claude Webflow Framer

It's not either/or.

Webflow

Official MCP server. Claude builds layouts, styles, components and CMS content in your existing project. You start the project in Webflow first.

Framer

Official MCP since 22 October. Claude creates and edits pages and CMS content on a separate branch. Nothing goes live until you publish.

The trade-off.

	Keep Webflow or Framer, prompt it	Build from scratch with Claude
Creative freedom	Within the tool's limits	Anything you can describe
Learning curve	Learn the tool, then prompt	Prompting from day one
Hosting	Included in the plan	You choose where it lives
Cost	Monthly plan per site	Free to low, depending on setup
Editing later	Visual editor, easy for clients	Through Claude, or by hand
Complexity	Low	Low to high, depending on setup

Part 2 · How we build

Four roles.

The brain

Claude chat

Thinks, decides, designs and prototypes. Writes the prompts for Code.

The hands

Claude Code

Builds the site, runs it locally, tests it, opens pull requests. Never decides design.

The source of truth

GitHub

Holds the design system and the site. If it's not in Git, it doesn't exist.

The window

GitHub Desktop

See changes, review and merge without the command line.

Chat thinks, Code builds, Git remembers.

The loop.

01 · Claude chat	02 · Claude chat	03 · GitHub Desktop	04 · Claude Code	05 · Claude Code	06 · GitHub
Design in chat	Update the system	Push to Git	Build with Code	Test on localhost	Pull request

← Not OK at stage 5? The report goes back to stage 1, in Claude chat.

Stage 1 of 6 · Claude chat

Design in chat

Prototype the page in the preview. Try versions. Fix phone, tablet and motion there.

Click a stage, or use Shift and the arrow keys. This loop stays on screen while you build.

System or page?

System

Reused across pages, or a rule. Goes into the design system first: tokens, components, storybook, skill.

Page

Used once. Stays on the page. A one-off that shows up twice becomes system.

Claude Code never decides design.

Every decision about colour, type, layout, copy and motion is made in Claude chat, and lives in the design system or the prototype.

The rule, already in your context file

- If something isn't defined, stop and ask
- Offer two or three options with a reason each
- Use only tokens and components, never a hard-coded value
- Test, then report every open question before a pull request

Part 3 · Where it lives

Our setup, and the easy way.

Our piece	Its job
GitHub	Design system and code
Cloudflare	Hosting and the domain
Resend	Emails and notifications
Supabase	Data: signups, form entries

Full control. Built for working with developers. Four accounts.

Tonight: Netlify

Hosting, forms and your own domain in one account. A real site, live, with a form that works.

Astro or Next.js?

Astro, for marketing sites

Fast, crawlable pages by default: finished HTML, no JavaScript unless a part needs it. That matters for AI search, because the big AI crawlers generally don't run JavaScript. The closest framework to plain HTML and CSS.

Next.js, when the product already is

It can produce equally crawlable pages, set up on purpose. One codebase and one team beats the speed gain. Prompting works the same either way. Tonight's kits are plain HTML: no framework needed.

Where to host.

	Built-in forms	Deploy without GitHub	Private repo in a GitHub org	Free for business sites	Tonight
Netlify	Yes	Yes, drag and drop	Pro, $20/month	Yes	Yes
Cloudflare Pages	Extra setup	Yes, drag and drop	Free	Yes	Hosting only
Vercel	No	Mostly via GitHub	Not checked	No, non-commercial	No
Render	No	Needs a repo	Not checked	Yes	No
GitHub Pages	No	Needs a repo	Not checked	Public repos only	No

Netlify is free until you work as a team. It wins on convenience; Cloudflare is the cheaper home for a team.

Part 4 · Get set up

Before we build: five accounts and apps.

1

Claude

Signed in at claude.ai. Workshop credits are on us.

2

GitHub

Free account at github.com/signup.

3

GitHub Desktop

Free app from desktop.github.com. Sign in with GitHub.

4

Claude Code

In the Claude desktop app, or in your terminal. Sign in with Claude.

5

Netlify

Free account at netlify.com. Sign up with your GitHub account.

Hands up when all five are done. We help anyone who's stuck.

Get the starter kit. Three ways, same result.

Easiest

Download ZIP

1. Open the link or scan the code
2. Code, then Download ZIP
3. Unzip and pick a kit: Paper, Clay or Signal

With the app

GitHub Desktop

1. File, then Clone repository
2. URL tab: paste the repo link
3. Choose a folder, then Clone

Command line

Terminal

```
git clone
https://github.com/hoomanab01/web-
style-guide
```

Does the same as the other two, typed.

github.com/hoomanab01/web-style-guide

The kit is your starting point, not your repository. Next, you make your own.

Two repositories, two jobs.

yourbrand-design-system

The design system

- Tokens, components and the storybook
- Page designs from Claude chat, in a prototypes folder
- The context file and the design skill

Where design decisions live. Claude chat works from here.

yourbrand-website

The website

- The pages Claude Code builds
- Branches: main is live, staging is the rehearsal
- Pull requests, each with a preview link

Where the site lives. Claude Code builds here; Netlify publishes it.

Design is decided in one. The website is built in the other.

Create both repositories.

yourbrand-design-system

1. GitHub Desktop: File, then New repository
2. Name: yourbrand-design-system, then Create
3. Copy into it from your kit: assets, storybook.html, SKILL.md, CLAUDE.md
4. Commit, then Publish repository

yourbrand-website

1. File, then New repository
2. Name: yourbrand-website, then Create
3. Copy into it from your kit: the pages, assets, netlify.toml and CLAUDE.md
4. Commit, then Publish repository

Keep them private if you like. Netlify and Claude can still reach them once connected.

Connect Claude to your GitHub.

Claude chat

1. Settings, then Connectors
2. Connect GitHub and allow your two repositories
3. In your Project you can now add files straight from yourbrand-design-system

Claude Code

1. Open Claude Code in your yourbrand-website folder
2. The first time it pushes, it asks you to sign in to GitHub
3. Sign in, and allow access

Chat reads the design system. Code builds the website. Both through GitHub.

Open Claude and set up your context.

1. In Claude, create a Project named after your website
2. Add to the Project, from yourbrand-design-system: CLAUDE.md, SKILL.md, assets and storybook.html
3. Paste the Project instructions on the right
4. Ask Claude to fill in CLAUDE.md with you, using the prompt below, and save it in both repositories

Project instructions

Follow CLAUDE.md and SKILL.md for every answer. Our Git: design system at github.com/[you]/yourbrand-design-system, website at github.com/[you]/yourbrand-website. Build only from the design system. Claude Code never makes a design decision; when something isn't defined, ask me first.

Fill in the context file

Help me fill in CLAUDE.md for my business. Ask me one question at a time: what I do, who it's for, what makes me different, and the links to my two repositories.

Turn on Anthropic's skills.

Skill	What it helps with tonight
frontend-design	Designed pages instead of generic ones: layout, type, motion
web-artifacts-builder	Bigger interactive pages with several parts
canvas-design	Visuals and posters, for moodboards and imagery
theme-factory	Trying colour and font themes quickly
skill-creator	Turning your design rules into a skill later

1. In Claude: Settings, then Capabilities
2. Turn on Skills
3. Turn on frontend-design first, then the others you want

Your kit's SKILL.md already sits in your Project, so Claude follows your design rules too.

Connect Netlify to yourbrand-website.

1. In Netlify: Add new project, then Import an existing project, then GitHub
2. Choose yourbrand-website
3. Check the settings on the right, then Deploy
4. Project configuration, Build and deploy, Branches: add staging as a branch deploy
5. Change the project name to yourbrand, so the address is yourbrand.netlify.app

Field	Our kits (plain HTML)
Branch to deploy	main
Base directory	empty
Build command	empty
Publish directory	.

The kits include netlify.toml, so these usually fill in by themselves.

Live site, staging, and preview links.

main

The live site

What your visitors see, at yourbrand.netlify.app. It changes only when you merge.

staging

A rehearsal copy

A second, private copy of the whole site at its own web address: staging--yourbrand.netlify.app. Same site, newer changes. Visitors never see it.

pull request

A preview link

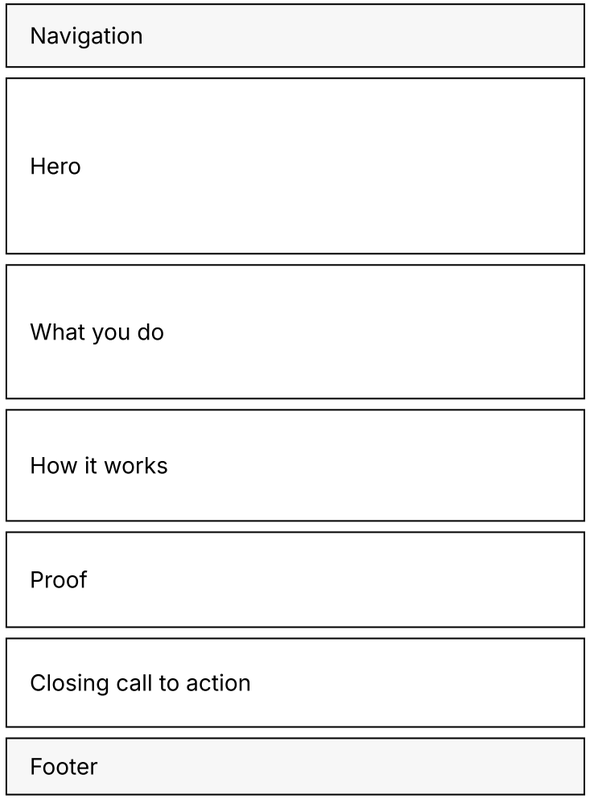
For every pull request, Netlify builds a temporary copy of the site with just that change and puts the link in the pull request. Click it, check it, then merge.

Check on a copy. Merge when it's right. Only then does the live site change.

Part 5 · Build

We build one page, the home page, all the way around the loop.

The home page, as a skeleton.



Section	What it needs
Navigation	Logo, four links, one button
Hero	One headline that says what you do. One line under it. One primary button.
What you do	Three things you offer, each with a title and one sentence
How it works	Three steps, from first contact to result
Proof	A real quote, a client logo or a number, with its source
Closing call to action	One line and one button: get in touch
Footer	Links, Impressum, privacy

Get inspired before you prompt.

1. Collect 5 to 10 references on Pinterest or from sites you love: layouts, type, motion, photography
2. Screenshot them
3. Drop the screenshots into your Claude chat
4. Say what you like about each one, in your own words

Turn references into direction

Here are references I like: [attach screenshots]. For each one, tell me what makes it work: layout, type, colour, motion, storytelling. Then suggest how I could bring that feeling to my home page, using my design system.

Make versions. See them. Pick one.

Three directions

Design three versions of my home page hero, using only my design system: 1. calm and editorial, 2. bold with motion, 3. visual storytelling as you scroll. Show them side by side so I can compare them. Then tell me which fits my audience best and why.

See them side by side

Ask for the versions as a design in Claude, so they sit next to each other on one canvas. Then ask for changes on the one you like.

A motion menu to ask for

- Headlines that rise word by word
- Sections that reveal as you scroll
- A sticky story: text stays, visuals change
- Hover states that respond, subtly

Stage 1. Design the home page in chat.

Prototype

Build my home page following the skeleton: navigation, hero, what I do, how it works, proof, closing call to action, footer. Use the direction we picked and only my design system. Show it to me, then show me the phone width.

Check it in the preview

- A stranger gets what you do in five seconds
- One clear next step
- Works on a phone
- Sounds like you, not a template

Stage 2. Update the design system.

System or page?

Look at the home page we just made. List every decision that should become part of the design system: new tokens, new components, new rules. Then give me the updated tokens.css, components.css, storybook.html and SKILL.md, and the final page as prototypes/home.html.

Save the files into your yourbrand-design-system folder, replacing the old ones.

Reused or a rule: system. Used once: page.

Stage 3. Push the design system to Git.

1. Open GitHub Desktop and choose yourbrand-design-system
2. Your changed files are listed on the left
3. Bottom left: a short summary, for example "Home page design and system v1"
4. Commit, then Push origin

Push origin means: send my saved work to GitHub.

The design and the prototype are now the source of truth. Claude Code builds only from this.

Stage 4. Hand over to Claude Code.

To Claude Code

Read CLAUDE.md. First, get the latest from github.com/[you]/yourbrand-design-system and update this website's tokens and components to match. Then build index.html from prototypes/home.html in that repository. Never make a design decision: if something isn't defined, stop and ask me with two or three options. Work on a new branch called home-page. When done, tell me what you built and every open question.

Open Claude Code in yourbrand-website

- Desktop app: choose the yourbrand-website folder
- Terminal: cd into the folder, then type claude

Questions from Claude Code? Answer them, or take them back to chat.

Stage 5. Test on localhost.

Run and test

Start a local server for this site and give me the address. Then run Lighthouse on the home page for mobile and desktop, check every breakpoint and every link, and report the scores and anything that breaks the design system.

You check

- Open the localhost address
- Phone width, tablet, desktop
- Motion feels like the prototype
- Every link and button works

Not OK? Back to chat

Here is Claude Code's report: [paste]. Which of these change the design system and which are page fixes? Write the next prompt for Claude Code.

Stage 6. Pull request, preview, merge.

To Claude Code

Everything works. Commit the changes on the home-page branch with a clear message, push it, and open a pull request to main with a summary of what changed and the test results.

1. Open the pull request on GitHub
2. Wait for the Netlify check, then open its Deploy Preview link
3. Click through the preview on your phone
4. Happy? Merge pull request. yourbrand.netlify.app updates in a minute

One page, all the way round. The next page is the same loop.

Homework

Homework 1. Make the contact form work.

1. Build the contact page around the loop, like the home page
2. In Netlify: Forms, then turn on form detection
3. Deploy again so Netlify finds the form
4. Forms, Form notifications: add an email to your inbox
5. Send yourself a message from the live site

GDPR starts here

A name and an email are personal data. Before real customers use it: a privacy notice, an Impressum, and a data processing agreement with every service that touches the form. Not legal advice.

Homework 2. Your own domain.

Buy it in Netlify

1. Domain management, Add a domain
2. Buy a new domain
3. Netlify connects it and turns on HTTPS

The easiest path.

Already own one

1. Add a domain you already own, Verify, Add domain
2. Add the record below where your DNS is managed
3. Add a TXT record too, if Netlify asks
4. Wait a few minutes, then HTTPS, Verify DNS configuration

You want	Add this record
www or a subdomain	CNAME, pointing to yourbrand.netlify.app
yourbrand.com	A record @, to the IP Netlify shows

- DNS on Cloudflare? Use DNS only, the grey cloud
- Never point a live domain at a test site
- Don't move nameservers on a domain you already use

Later: do you need a CMS?

Option	How it works	Good for
Claude is your CMS	"Add a blog post about X." Claude edits the files.	Solo founders, small sites
A Git-based CMS	An editor at yoursite.com/admin that saves to your repo. Decap, Sveltia, Pages CMS, TinaCMS.	Small teams, clients who want a simple editor
A headless CMS	Content lives in its own app; your site pulls it in at build time. Sanity, Storyblok, Contentful.	Teams publishing a lot

If you're the only editor and happy asking Claude, you don't need one yet.

Close

Stuck? Ask Claude.

To understand	To do it	When stuck
Explain [Git / a pull request / DNS] like you're talking to a third grader. Then give me one example from building a website.	I want to [connect my domain to Netlify]. Walk me through it step by step. One step at a time, and wait until I say done.	Here's what I see on my screen: [paste the error]. What does it mean and what do I do next? Keep it simple.

Asking is the skill. Every word from tonight is in GLOSSARY.md.

Close

Show and tell, then Monday.

Show and tell

Your links on the big screen. Three or four of you walk us through your site and the version you picked.

Monday

Homework: the contact form and your own domain. Then the next page, around the same loop. GUIDE.md walks you through every stage.

Close

Walk in with an idea.
Walk out with a live link.

github.com/hoomanab01/web-style-guide

Hooman Abbasi · Raise Studio